EVIDENCE-FIRST PORTFOLIO

AI Systems Evidence Dossier

Local agent infrastructure, routing, memory, voice, remote command loops, and evaluator loops.

Generated 2026-06-17

Remote WhatsApp-to-Agent Command Loop

Why I built it: A phone should be able to issue work to a home computer without turning the system into an unsafe direct shell. The design had to translate casual WhatsApp messages or audio into structured work orders, run them through a tool-enabled agent side, judge completion, and only then report back.

What I built / evidence: Implemented across OpenClaw watcher, MiguelAgent, EXOVIUM core, WA task manager, cognition routing, and architecture notes. Source notes document immediate ACK behavior, quality-loop thresholding, model routing, audio transcription, and two successful end-to-end tests.

Claim boundary: Prototype/evolved system. Some EXOVIUM pieces were later superseded by Miguel Core. Private channel identifiers and credentials are excluded.

Remote WhatsApp-to-Agent Command Loop - architecture

I include the architecture as a workflow because the value is in the sequence, not in a single model response.

- WhatsApp message or audio arrives through OpenClaw.
- A watcher normalizes inbound events, handles media, deduplicates repeated messages, and dispatches work.
- MiguelAgent interprets the request into goal, complexity, work order, success criteria, and model hint.
- The execution side uses EXOVIUM/Miguel tooling and model routing.
- A quality loop evaluates the result and can ask for an improved work order before replying.
- The final response is formatted and sent back through WhatsApp.

Remote WhatsApp-to-Agent Command Loop - what I can show

This is the clearest Secure AI signal: remote natural-language instructions became structured work orders, then passed through tool execution and evaluation before a user-facing reply.

- Independent design of a phone-to-computer agent workflow before relying on a managed product workflow.
- Separation of front-agent understanding/evaluation from back-agent execution.
- Ability to turn vague mobile input into structured work orders with success criteria.
- Awareness that autonomous work needs progress, interruption, deduplication, and post-work evaluation.
- Remote control is an authority problem, not just an interface problem.
- Acknowledge fast, execute slowly, and report only after evaluation.
- Audio, chat, browser events, model output, and tool execution all need different failure handling.
- Credentials and channel identifiers must be isolated from public documentation.

Remote WhatsApp-to-Agent Command Loop - evidence cluster and next hardening

The public version uses sanitized source clusters rather than raw source dumps, because raw folders would expose private paths, credentials, unfinished experiments, and noise.

- Workflow notes describing WhatsApp event detection, audio transcription, structured work orders, model hints, and quality-loop thresholds.
- Pipeline notes mapping WhatsApp -> OpenClaw gateway -> watcher -> MiguelAgent orchestration -> execution/evaluation/formatting -> WhatsApp reply.
- Python watcher and MiguelAgent modules showing event parsing, media handling, deduplication, work-order execution, evaluator loop, and response formatting.
- Decision notes recording immediate acknowledgement, smart routing, and successful end-to-end test runs.
- Replace private ad-hoc credentials with explicit scoped tokens.
- Add command-class permissions and confirmation thresholds.
- Store execution traces in a reviewer-safe audit log.
- Retest the end-to-end path after Miguel Core consolidation.

Miguel Core Unified Daemon

Why I built it: Early systems had separate daemons, command scripts, and services. Miguel Core consolidated the control surface into a local Python daemon that could coordinate chat, streaming, voice, memory, model tiers, orchestration, state, and tool endpoints.

What I built / evidence: Central file `miguel_core.py` documents the consolidation goal and exposes the route surface. `orchestra.py` implements async worker delegation to Opus, Codex, GPT-style workers through a subprocess layer with timeouts, cancellation, status, and stats.

Claim boundary: Local experimental control plane, not a hardened commercial security product.

Miguel Core Unified Daemon - architecture

I include the architecture as a workflow because the value is in the sequence, not in a single model response.

- One daemon replaces fragmented service surfaces.
- Routes cover chat/streaming, voice, state, vault, knowledge, prediction, cascade, agents, metacognition, orchestra, macOS/tool endpoints, and conversations.
- Imports connect memory broker, CASS pipeline, vault watcher, prediction, metacognition, state graph, orchestra, and macOS bridge.
- Startup/handover scripts define model routing, state probes, boot checks, and autonomy rings.

Miguel Core Unified Daemon - what I can show

This shows system-level building rather than model use: interfaces, state, memory, tools, routing, health, orchestration, and operating boundaries were treated as one control plane.

- Ability to consolidate fragmented prototypes into a local control plane.

- Understanding of agent systems as services with routes, state, health, models, memory, and tools.
- Experience with broad API surfaces while still thinking about boot probes and autonomy rings.
- Practical path from scripts and separate daemons toward one observable operating layer.
- Agent reliability depends on process state, permissions, memory, model availability, and user intent.
- A route existing is not the same as a route being production-hardened.
- Local-first systems still need explicit observability and failure boundaries.
- Autonomy rings are a useful design language for separating safe actions from user-only actions.

Miguel Core Unified Daemon - evidence cluster and next hardening

The public version uses sanitized source clusters rather than raw source dumps, because raw folders would expose private paths, credentials, unfinished experiments, and noise.

- Miguel Core daemon source documenting the consolidation of earlier daemon/command surfaces into one local control plane.
- Route map covering chat, streaming, voice, state, vault, knowledge, prediction, cascade, agents, orchestra, macOS/tool control, conversations, and health surfaces.
- Orchestra worker module implementing delegated async agent tasks, timeouts, cancellation, status, and result capture.
- Startup and handover documents showing boot probes, routing policies, quality gates, operating modes, and autonomy rings.
- Create a public demo mode with secrets disabled.
- Add endpoint-level permission metadata.
- Add machine-readable health reports for all major modules.
- Shrink the public API surface into documented stable interfaces.

Multi-Model Cascade Router

Why I built it: Agent systems fail when all tasks are pushed through one model. The router treats models as resources with strengths, costs, quotas, cooldowns, and failure modes.

What I built / evidence: `ai_router.py`, `cascade.py`, and `litellm_config.yaml` define model registries, routing strategies, tiering, quota tracking, fallback, race/cancel behavior, and budget-aware escalation.

Claim boundary: Cost-reduction claims should be treated as design goals unless fresh logs verify exact percentages.

Multi-Model Cascade Router - architecture

I include the architecture as a workflow because the value is in the sequence, not in a single model response.

- Task strategy selects speed, quality, free, code, creative, bulk, or local routing.
- Local models are tried before free APIs and paid tiers where appropriate.
- Quota/cooldown tracking blocks bad or unavailable routes.
- Quality gates decide whether to escalate.

- Free API models can be raced in parallel and slower calls cancelled after a sufficient answer.
- Outcomes are logged for later learning.

Multi-Model Cascade Router - what I can show

This shows model governance in practice: models are resources with availability, cost, quality, quota, privacy, and failure characteristics.

- Model selection was treated as an engineering system rather than a preference.
- The design accounts for cost, quotas, rate limits, cooldowns, historical quality, and fallback.
- The router connects local/offline models with API/CLI providers.
- Parallel racing and cancellation were considered for free-model tiers.
- A strong agent stack should degrade gracefully when models fail.
- Quality gates should decide escalation, not model status alone.
- Cost and privacy are part of capability routing.
- Outcome logging is the seed of empirical model governance.

Multi-Model Cascade Router - evidence cluster and next hardening

The public version uses sanitized source clusters rather than raw source dumps, because raw folders would expose private paths, credentials, unfinished experiments, and noise.

- AI router source with provider registry for Gemini, Groq, Cerebras, OpenRouter, DeepSeek, Together, Ollama/local models, and LiteLLM-backed calls.
- Cascade source implementing local/free/paid tiers, quota and cooldown tracking, quality gates, free-model racing, cancellation, paid-budget blocking, and outcome logging.
- LiteLLM configuration separating local T0, free API T1, and paid T2 tiers.
- Architecture notes describing routing principles, quality scoring, budget handling, and quota-aware fallbacks.
- Run a fresh benchmark across representative task classes.
- Publish aggregate cost/latency/quality results without provider secrets.
- Add policy labels for private, public, cheap, fast, and high-stakes tasks.
- Use the router as the basis for a secure-agent evaluation project.

Memory, Vault, CASS, and Smart Memory MCP

Why I built it: Long-running agents need more than a large context window. They need raw memory, compressed memory, procedural learning, contradiction handling, retrieval, and a way to avoid acting on stale facts.

What I built / evidence: `cass_pipeline.py`, `memory_broker.py`, vault-watcher design notes, and `smart-memory-mcp/server.js` show schemas, scoring, contradiction logic, TF-IDF retrieval, recency/usefulness metadata, and MCP tool declarations.

Claim boundary: Presented as practical memory infrastructure, not proof of human-like consciousness.

Memory, Vault, CASS, and Smart Memory MCP - architecture

I include the architecture as a workflow because the value is in the sequence, not in a single model response.

- CASS keeps episodic raw records, structured working memory, and procedural rules with confidence/decay.
- Memory Broker broadcasts high-value or central facts across memory systems and flags contradictions.
- Vault watcher design handles file-change detection, incremental reindexing, graph updates, staleness propagation, and forgetting scores.
- Smart Memory MCP provides a simple local protocol component for learn/recall/stats/patterns/suggestions.

Memory, Vault, CASS, and Smart Memory MCP - what I can show

This shows that long-running agents need memory governance: raw records, summaries, procedural learning, contradictions, staleness, and retrieval metadata.

- Memory was designed as structure, not just as a folder of notes.
- The system distinguishes raw episodes, compressed working memory, and procedural rules.
- Contradiction and staleness were treated as first-class risks.
- A small MCP component makes the memory idea portable across agent clients.
- Agents can become dangerous or useless when they act on stale summaries.
- Raw memory should remain available when summaries become distorted.
- Contradictions should be preserved and surfaced instead of overwritten.
- Retrieval scoring needs more than text similarity; usefulness and recency matter.

Memory, Vault, CASS, and Smart Memory MCP - evidence cluster and next hardening

The public version uses sanitized source clusters rather than raw source dumps, because raw folders would expose private paths, credentials, unfinished experiments, and noise.

- Memory Broker source showing broadcast schema, write logs, weight/centrality scoring, and lightweight contradiction detection.
- CASS source implementing episodic raw memory, working summaries, procedural rules, confidence, decay, and SQLite persistence.
- Vault watcher design notes covering file-change detection, incremental indexing, graph updates, staleness propagation, and forgetting score.
- Smart Memory MCP source exposing learn/recall/stats/pattern/suggestion tools with retrieval metadata.
- Add a public synthetic-memory demo.
- Benchmark retrieval and contradiction detection on non-private data.
- Expose memory provenance in every agent response.
- Add explicit redaction rules for private vault content.

Voice Interface and Spoken Tool Use

Why I built it: Voice control is powerful because it lowers friction, but spoken commands are ambiguous and can trigger tools. The system needed tool routing, muting, fallbacks, destructive-command filtering, and a privacy-forward plan.

What I built / evidence: `voice.py` contains Gemini Live config, sample-rate handling, tool declarations, dispatcher, vault tools, deep-think route, shell guard, reconnect/fallback loop, and VoiceBridge experiments. Planning notes define latency targets, privacy red lines, and acceptance criteria.

Claim boundary: Wake-word/macOS app features should be described as planned or partially integrated unless retested.

Voice Interface and Spoken Tool Use - architecture

I include the architecture as a workflow because the value is in the sequence, not in a single model response.

- Python terminal voice interface streams microphone audio into Gemini Live.
- The Live config exposes tools for vault search/read/write, deep thinking through Miguel Core, brain status, context, and constrained shell execution.
- Dangerous shell patterns are blocked and command execution is timeout-limited.
- A later design adds wake-word, local STT, hybrid TTS, barge-in, and privacy boundaries.

Voice Interface and Spoken Tool Use - what I can show

This shows voice as a safety-sensitive command surface, not just a pleasant chat interface.

- Voice was integrated with tools, not just used as speech-to-chat.
- The interface can route from spoken input into vault operations, daemon calls, and constrained shell execution.
- The design includes muting, fallbacks, destructive-command blocking, and privacy plans.
- The later pipeline notes show awareness of wake-word, local STT, TTS, and barge-in constraints.
- Voice commands should be treated as high-friction-to-confirm, low-friction-to-issue.
- Always-listening systems require local processing and explicit privacy boundaries.
- Barge-in is a safety feature because it lets the user stop the system quickly.
- A beautiful voice is irrelevant if authority and privacy are unclear.

Voice Interface and Spoken Tool Use - evidence cluster and next hardening

The public version uses sanitized source clusters rather than raw source dumps, because raw folders would expose private paths, credentials, unfinished experiments, and noise.

- Voice source implementing microphone streaming, Gemini Live configuration, tool declarations, dispatcher, vault tools, deep-think route, constrained shell, reconnect, and fallback behavior.
- Voice pipeline notes defining wake-word, local STT, hybrid TTS including local/provider voices, barge-in, latency budget, and privacy red lines.
- VoiceBridge notes separating browser/orb STT work from the local daemon response path.

- Safety-relevant source sections around mute behavior, destructive-command filtering, timeouts, and tool dispatch.
- Retest browser/microphone E2E with a clean public demo.
- Separate safe voice tools from confirmation-required tools.
- Add transcript-level audit and user correction paths.
- Measure latency rather than relying on target budgets.

ANIMA Experimental Kernel

Why I built it: Speculative cognition ideas often remain untestable prose. ANIMA turns a set of cognitive/consciousness-inspired primitives into a Python package with state, metrics, tests, benchmarks, and evidence-status notes.

What I built / evidence: Local test run during rebuild: `python3 -m pytest -q` in `anima-kernel` returned 446 passed tests with one config warning. Benchmark evidence notes say current results support architecture/testbed value, not proof of consciousness.

Claim boundary: Never present as proof of artificial consciousness. Use as an ambitious, testable experimental architecture.

ANIMA Experimental Kernel - architecture

I include the architecture as a workflow because the value is in the sequence, not in a single model response.

- Kernel/state/type modules.
- Primitives such as qualia, engram, valence, nexus, impulse, trace, mirror, and flux.
- Temporal substrate and metric layers.
- Benchmarks and ablation methodology.
- Evidence status file that overrides over-strong claims.

ANIMA Experimental Kernel - what I can show

This shows ambition with discipline: speculative cognitive architecture converted into code, tests, benchmarks, and explicit evidence limits.

- A speculative cognitive idea was converted into a package with modules, tests, benchmarks, and methodology.
- The project can be discussed honestly because evidence-status notes bound the claims.
- Local tests passing at scale show implementation substance.
- The architecture gives a playground for memory, temporal continuity, and self-modeling ideas.
- Ambitious terms need stronger evidence than ordinary software claims.
- Benchmark files should override README optimism.
- Non-significant results still matter if they shape the next experiment.
- A serious project can be valuable even when its strongest philosophical claim is not proven.

ANIMA Experimental Kernel - evidence cluster and next hardening

The public version uses sanitized source clusters rather than raw source dumps, because raw folders would expose private paths, credentials, unfinished experiments, and noise.

- ANIMA Python package with kernel, state, primitives, temporal modules, metrics, bridge/shell modules, tests, and benchmarks.
- Evidence-status and methodology files that bound claims and identify current benchmark limitations.
- Local rebuild test run with 446 passing tests and one configuration warning.
- Architecture notes connecting ANIMA to memory, temporal continuity, self-modeling, and Miguel integration.
- Create a clean benchmark report with baselines and statistical method.
- Separate philosophy, architecture, and measured results in public docs.
- Run ablations after fixing baseline limitations.
- Use ANIMA as an experimental module, not as the main Fellowship claim.